Every field in a document can have one or more associated types based on the business and data requirements. Table 4.1 provides a list of common data types with some examples.

Table 4.1 Common data types with examples

Type	Description	Examples
`text`	Represents textual information (such as string values); unstructured text	Movie title, blog post, book review, log message
`integer, long, short, byte`	Represents a number	Number of infectious cases, flights canceled, products sold, book's rank
`float, double`	Represents a floating-point number	Student's grade point average, moving average of sales, reviewer average ratings, temperature
`boolean`	Represents a binary choice: true or false	Is the movie a blockbuster? Are COVID cases on the rise? Has the student passed the exam?
`keyword`	Represents structured text: text that must not be broken down or analyzed	Error codes, email addresses, phone numbers, Social Security numbers
`object`	Represents a JSON object	(In JSON format) employee details, tweets, movie objects
`nested`	Represents an array of objects	Employee address, email routing data, a movie's technical crew

As you can imagine, this is not a comprehensive list of data types. This book is written for version 8.6, and nearly 40 data types are defined by Elasticsearch. Elasticsearch defines types microscopically in some cases to optimize search queries. For example, `text` types are further classified into more specific types such as `search_as_you_type`, `match_only_text`, `completion`, `token_count`, and others.

Elasticsearch also tries to group data types in families to optimize space and performance. For example, the `keyword` family has `keyword`, `wildcard`, and `constant_keyword` data types. Currently this is the only family group; we can expect more in the near future.

Single field with multiple data types

In programming languages like Java and C#, we can't define a variable with two different types. However, there is no such restriction in Elasticsearch. Elasticsearch is pretty cool when it comes to representing a field with multiple data types, allowing us to develop the schema the way we want to.

For example, we may want the author of a book to be both a `text` type and a `keyword` type. Each field has a specific characteristic, and keywords will not be analyzed, meaning the field is stored as is. We can also have more types—for example, the `author` field can be declared as a `completion` type in addition to the other two types.